

Contents

□ EMACS IDE — Roadmap de Bibliotecas e Integração	2
Outline Completo Extraído das Fontes em ~/emacs	2
□ Sumário	2
Outline I: Bibliotecas por Categoria	2
□ 1. Fundações do Ecossistema	2
□ 2. Interface e UX	3
⇒ 3. Edição e Movimento	4
□ 4. Busca e Navegação	6
□ 5. Code Intelligence (LSP)	8
□ 6. Autocompletar	9
□ 7. Org Mode e Conhecimento	11
□ 7. Git e Versionamento	13
□ 8. AI e Integração	14
□ 9. Terminal e Shell	16
□ 10. Formatação e Estilo	17
□ 11. Web e Crawling	18
□ 12. Segurança	19
□ 13. Modos Maiores Especiais	19
□ 14. Visualização e Dados	21
□ 15. UX e Produtividade	23
□ 16. Buffers Especiais e IRs	24
□ 17. Integração Python (EPC)	25
□ 18. Testing e Debug	26
□ 19. Funcionalidades Avançadas (Futuro)	27
□ 20. Dependências Python (Instalar Separadamente)	28
Outline II: Relacionamentos entre Bibliotecas	29
□ 1. Cadeia de Dependência Funcional	29
□ 2. Fluxos de Dados Entre Bibliotecas	33
□ 3. Map de Ativações e Mutual Indexing	34
□ 4. Arquitetura de Comunicação	35
□ 5. Grupos de Co-instalação (Pacotes Recomendados)	35
Plano de Implementação Pragmático	37
Fase 1: Fundação (Dia 1-2)	37
Fase 2: Edição e Navegação (Dia 3-4)	37
Fase 3: Code Intelligence (Dia 5-6)	37
Fase 4: Git e Versionamento (Dia 7)	37
Fase 5: Org e Conhecimento (Dia 8-10)	38
Fase 6: AI Integration (Dia 11-14)	38
Fase 7: Polish e Avançado (Dia 15+)	38
Priorização por Impacto	38

□ EMACS IDE — Roadmap de Bibliotecas e Integração

Outline Completo Extraído das Fontes em ~/emacs

“A excelência de um sistema não está em suas partes isoladas, mas na harmonia com que elas colaboram.”

— Filosofema Zen aplicado à engenharia de software

□ Sumário

1. Outline I: Bibliotecas por Categoria
 2. Outline II: Relacionamentos entre Bibliotecas
 3. Plano de Implementação Pragmático
-

Outline I: Bibliotecas por Categoria

□ 1. Fundações do Ecossistema

□ use-package + straight.el

Emoji: 📦

Rationale: Gerenciador declarativo de configuração que organiza código em blocos reutilizáveis, com lazy-loading integrado. Elimina startup lento com :defer automático.

Filosofema: *“A ordem precede a função. Um sistema bem organizado antecipa seu próprio uso.”*

```
;; Configuração canônica
(straight-use-package 'use-package)
(use-package nome-do-pacote
  :ensure t                ;; baixa se não existir
  :defer t                 ;; carrega sob demanda
  :config (setup-code)    ;; roda após carregamento
  :bind (("C-c k" . comando)) ;; atalhos
```

□ which-key

Emoji: 🗝️

Rationale: Exibe popup com atalhos disponíveis após digitar prefixo — elimina memorização de keybindings. Essencial para descobrir funcionalidades sem consultar documentação.

Filosofema: *“O conhecimento que não se revela é como a luz sob o alqueire.”*

```
(use-package which-key
  :ensure t
  :config
  (which-key-mode +1)
  (setq which-key-idle-delay 0.3))
```

▣ posframe

Emoji: 📄

Rationale: Renderiza buffers pop-up em frames posicionáveis — habilita tooltips, previews e menus flutuantes sem dependência de bibliotecas externas. Base para peek-definition e flymake popups.

Filosofema: *“A interface perfeita é aquela que aparece quando necessária e desaparece quando não é.”*

```
(use-package posframe
  :ensure t
  :config
  (setq posframe-mouse-mode t
        posframe-fade-mode t))
```

▣ 2. Interface e UX

▣ modus-themes

Emoji: 📄

Rationale: Tema acessível com suporte a claro/escuro, contrastado WCAG AAA. Inclui variante para daltonismo — prioriza legibilidade sobre estética.

Filosofema: *“Verdadeira beleza é clareza. A interface que não se nota é a interface perfeita.”*

```
(use-package modus-themes
  :ensure t
  :config
  (setq modus-themes-bold-constructs t
        modus-themes-mixed-fonts t)
  (load-theme 'modus-vivendi t))
```

▣ doom-modeline

Emoji: 📄

Rationale: Modeline minimalista com segmentos plugáveis — git branch, LSP status, flymake errors. Reduz scroll mental ao manter informações de estado sempre visíveis.

Filosofema: *“Cada dado visível economia uma pergunta. Cada pergunta interrompida é uma tarefa inacabada.”*

```
(use-package doom-modeline
  :ensure t
  :config
  (doom-modeline-mode +1)
  (setq doom-modeline-height 25))
```

▢ all-the-icons

Emoji: 🐱

Rationale: Provedor de ícones para treemacs, neotree, diminish. Usa fontes Nerd Fonts — visualização rica sem overhead de imagens.

Filosofema: *“Um ícone vale mil palavras — mas apenas quando o contexto exige tradução visual.”*

```
(use-package all-the-icons
  :ensure t
  :config
  (when (member "0xProto" (font-family-list))
    (setq all-the-icons-font-family "0xProto")))
```

▢ helpful

Emoji: 🐱

Rationale: Substitui describe-function com documentação renderizada — hyperlinks para fontes, referências croass, examples de uso. Transforma introspecção em experiência fluida.

Filosofema: *“Explicar é entender duas vezes. O helpful força a clareza que a pressa ignora.”*

```
(use-package helpful
  :ensure t
  :config
  (global-set-key (kbd "C-h f") #'helpful-callable)
  (global-set-key (kbd "C-h v") #'helpful-variable))
```

⇒ 3. Edição e Movimento

▢ avy

Emoji: 🐱

Rationale: Salto visual para qualquer char/word/line visível — combina speed com precisão. Substitui isearch para navegação em múltiplas

janelas. Superiores ao ace-jump em UX.

Filosofema: *“O atalho mais direto é o que os olhos já traçaram.”*

```
(use-package avy
  :ensure t
  :bind
  (("C-c j j" . avy-goto-char-2)
   ("C-c j l" . avy-goto-line)
   ("C-c j w" . avy-goto-word-1)))
```

☐ windmove

Emoji: ☐

Rationale: Navegação entre janelas via S-<arrows> — elimina necessidade de C-x o. Integração natural com avy para quem já usa setas.

Filosofema: *“Janelas são portais; setas são bússolas.”*

```
(use-package windmove
  :config
  (windmove-default-keybindings)
  (setq windmove-wrap-around t))
```

☐ smartparens

Emoji: ☐

Rationale: Gerenciamento automático de pares — wrap, unwrap, navigate sexp. Reduz erros de parênteses em 90% em arquivos Lisp.

Filosofema: *“O par que se fecha sozinho é o par que se abre com propósito.”*

```
(use-package smartparens
  :ensure t
  :config
  (smartparens-global-mode +1)
  (sp-local-pair 'emacs-lisp-mode "" nil :actions nil))
```

☐ rainbow-delimiters

Emoji: ☐

Rationale: Colore delimitadores aninhados por nível — facilita leitura de sexps profundos em Lisp/Org/JSON. Feedback visual imediato de estrutura.

Filosofema: *“Profundidade que se vê é profundidade que se compreende.”*

```
(use-package rainbow-delimiters
  :ensure t
```

```
:config
(add-hook 'prog-mode-hook #'rainbow-delimiters-mode))
```

□ 4. Busca e Navegação

□ consult

Emoji: ☹️

Rationale: Suite de busca fuzzy unificada — ripgrep, imenu, line, buffer. Preview inline elimina necessidade de abrir arquivos para verificar conteúdo. Integra com embark.

Filosofema: *“Encontrar sem procurar é sabedoria; encontrar rápido é tecnologia.”*

```
(use-package consult
  :ensure t
  :bind
  (("C-c s r" . consult-ripgrep)
   ("C-c s i" . consult-imenu)
   ("C-c s l" . consult-line)))
```

□ embark

Emoji: ☹️

Rationale: Menu contextual universal — click direito em qualquer alvo oferece ações relevantes. Atua como hub de ações sem necessidade de menus fixos.

Filosofema: *“O contexto revela as ações possíveis. O embark as materializa.”*

```
(use-package embark
  :ensure t
  :bind
  (("C-." . embark-act)
   ("C-h B" . embark-bindings))
:config
(setq embark-action-indicator
  (lambda (window _ _ _) (posframe-show "*Embark*" :string "...")))
```

□ projectile

Emoji: ☹️

Rationale: Reconhecimento de projetos git/hg/dir-locals. Navegação de arquivos por projeto, grep, replace em batch. Base para ferramentas de pro-

jeto como treemacs.

Filosofema: *“Todo código vive em contexto. O projectile honra esse contexto.”*

```
(use-package projectile
  :ensure t
  :config
  (projectile-mode +1)
  (setq projectile-switch-project-action #'projectile-dired)
  (define-key projectile-mode-map (kbd "C-c p") 'projectile-command-map))
```

▣ treemacs

Emoji: 🌳

Rationale: File tree lateral com integração LSP —🔍 arquivos por git status, erros, referências. Atualização automática via filewatch.

Filosofema: *“A árvore que se vê facilita a floresta que se perde.”*

```
(use-package treemacs
  :ensure t
  :config
  (treemacs-follow-mode t)
  (treemacs-filewatch-mode t)
  (treemacs-git-mode 'deferred)
  :bind
  (("C-c t t" . treemacs)
   ("C-c t f" . treemacs-find-file)))
```

▣ imenu-list

Emoji: 📋

Rationale: Outline lateral de símbolos do buffer atual. Auto-resize conforme conteúdo. Complemento visual ao xref para estrutura de arquivo.

Filosofema: *“O índice que se atualiza é o índice que se confia.”*

```
(use-package imenu-list
  :ensure t
  :config
  (setq imenu-list-auto-resize t
        imenu-list-focus-after-activation t)
  :bind ("C-c l" . imenu-list-smart-toggle))
```

▢ dumb-jump

Emoji: 🐢

Rationale: Fallback para goto-definition sem LSP — usa regex heurístico sobre git-grep. Funciona em qualquer linguagem sem setup de servidor.

Filosofema: *“Quando o mapa falha, a intuição heurística prevalece.”*

```
(use-package dumb-jump
  :ensure t
  :config
  (add-hook 'xref-backend-functions #'dumb-jump-xref-activate)
  (setq dumb-jump-prefer-searcher 'rg))
```

▢ 5. Code Intelligence (LSP)

▢ eglot

Emoji: 🐍

Rationale: Client LSP nativo Emacs 29+ — semantic tokens, inlay hints, find definitions/references. Elimina dependência de lsp-mode para linguagens suportadas.

Filosofema: *“O compilador que conversa é o compilador que orienta.”*

```
(use-package eglot
  :ensure t
  :hook
  ((python-ts-mode . eglot-ensure)
   (typescript-ts-mode . eglot-ensure)
   (js-ts-mode . eglot-ensure))
  :config
  (add-to-list 'eglot-server-programs
    '(python-ts-mode . ("pyright-langserver" "--stdio")))
  (setq eglot-inlay-hints-mode t
        eglot-autoshutdown t))
```

▢ treesit-auto

Emoji: 🌳

Rationale: Auto-instalação e carregamento de grammar tree-sitter. Parser incremental para linguagens modernas — 10x mais rápido que regexp-based. Base para navegação AST nativa.

Filosofema: *“A estrutura que se parseia é a estrutura que se compreende.”*

```
(use-package treesit-auto
  :ensure t
  :config
```



```
(global-treesit-auto-mode +1)
(setq treesit-auto-install 'prompt))
```

▢ flymake

Emoji: 🐛

Rationale: Linter nativo — integra com LSP diagnostics, flycheck. Exibe errors inline sem necessidade de ferramentas externas.

Filosofema: *“O erro visível é o erro que se corrige.”*

```
(use-package flymake
  :config
  (global-flymake-mode nil) ;; ativar por modo
  :bind
  (("M-g n" . flymake-goto-next-error)
   ("M-g p" . flymake-goto-prev-error)))
```

▢ flymake-vale

Emoji: 🐛

Rationale: Flymake backend para Vale — linting de prosa técnica. Verifica consistência de terminologia, estilo de escrita em documentos.

Filosofema: *“A documentação que se revisa é a documentação que se respeita.”*

```
(use-package flymake-vale
  :ensure t
  :hook
  ((markdown-mode . flymake-vale-setup)
   (org-mode . flymake-vale-setup)))
```

▢ 6. Autocompletar

▢ company-mode

Emoji: 🐛

Rationale: Framework de autocompletar plugável — backends para LSP, Yasnippet, words. Popup com documentação inline e source grouping.

Filosofema: *“Sugestão que vem antes da pergunta antecipa a necessidade.”*

```
(use-package company
  :ensure t
  :config
  (global-company-mode +1))
```

```
(setq company-idle-delay 0.2
      company-minimum-prefix-length 2
      company-tooltip-align-annotations t))
```

▢ company-statistics

Emoji: 📊

Rationale: Ordenação de completions por frequência de uso — adapta ao seu padrão de typing. Reduz cliques ao promover opções recentes.

Filosofema: *“O hábito molda a interface; a interface molda o hábito.”*

```
(use-package company-statistics
  :ensure t
  :config
  (company-statistics-mode +1)
  (setq company-statistics-size 100))
```

▢ yasnippet

Emoji: 📄

Rationale: Sistema de snippets expandíveis — placeholder, mirror, export. Milhares de snippets pré-definidos para linguagens comuns.

Filosofema: *“O template que se repete é o template que se automatiza.”*

```
(use-package yasnippet
  :ensure t
  :config
  (yas-reload-all)
  (yas-global-mode +1)
  (setq yas-snippet-dirs '("~/emacs.d/snippets")))
```

▢ copilot

Emoji: 🤖

Rationale: Autocompletar via GPT-4/Codex — sugestões de bloco completo. Tab para aceitar, diferencia de company.

Filosofema: *“A IA que completa antecipa o pensamento.”*

```
(use-package copilot
  :ensure t
  :config
  (global-copilot-mode +1)
  :bind
```

```
(( "C-c i" . copilot-accept-completion)
  ("C-c ]" . copilot-next-completion)))
```

□ 7. Org Mode e Conhecimento

□ org-roam

Emoji: 🧠

Rationale: Grafo de conhecimento emergido — links bidirecionais entre notas. Capture templates, backlinks, org-roam-ui para visualização.

Filosofema: *“O saber que se conecta é o saber que se retém.”*

```
(use-package org-roam
  :ensure t
  :config
  (setq org-roam-directory "~/org-roam/"
        org-roam-db-location "~/.emacs.d/org-roam.db")
  (org-roam-db-autosync-mode +1)
  :bind
  (( "C-c n f" . org-roam-node-find)
    ("C-c n i" . org-roam-node-insert)
    ("C-c n c" . org-roam-capture)
    ("C-c n l" . org-roam-buffer-toggle)))
```

□ org-roam-ui

Emoji: 🌐

Rationale: Interface web para visualizar grafo org-roam — D3.js force-directed graph. Follow mode sincroniza com nó atual.

Filosofema: *“O grafo que se vê é o grafo que se entende.”*

```
(use-package org-roam-ui
  :after org-roam
  :config
  (setq org-roam-ui-sync-theme t
        org-roam-ui-follow t
        org-roam-ui-update-on-save t
        org-roam-ui-open-on-start nil))
```

□ org-brain

Emoji: 🧠

Rationale: Mapa conceitual hierárquico — parent/child/friend para con-

ceitos. Visualização radial de ontologia. Mais determinístico que Roam.

Filosofema: *“O brain que se estrutura é o brain que se pensa.”*

```
(use-package org-brain
  :ensure t
  :config
  (setq org-brain-visualize-default-choices 'all)
  :bind
  (("C-c b" . org-brain-visualize)
   ("C-c B" . org-brain-goto-current)))
```

□ org-super-agenda

Emoji: 📅

Rationale: Views customizáveis de agenda — agrupa por projeto, contexto, prazo. Filtros avançados sem dependência de external tools.

Filosofema: *“A agenda que se personaliza é a agenda que se cumpre.”*

```
(use-package org-super-agenda
  :ensure t
  :config
  (org-super-agenda-mode +1)
  (setq org-super-agenda-groups
    '(:name "Hoje" :time-grid t :scheduled today)
      (:name "Projetos" :tag "projeto")
      (:name "Decisões" :tag "decisão"))))
```

□ org-transclusion

Emoji: 📄

Rationale: Inclui conteúdo de outros buffers inline — blocks fonte, headlines. Mantém synced sem duplicação.

Filosofema: *“O conteúdo que se reutiliza é o conteúdo que se respeita.”*

```
(use-package org-transclusion
  :ensure t)
```

□ org-download

Emoji: 📷

Rationale: Drag-and-drop de imagens para org-mode — screenshots, clipboard. Salva em diretório configurável com nomes padronizados.

Filosofema: *“A imagem que se arrasta é a imagem que se incorpora.”*

```
(use-package org-download
  :ensure t
  :config
  (setq org-download-method 'directory
        org-download-image-dir "~/org-images/")
  (org-download-enable))
```

🔗 org-frakti

Emoji: 🗒️

Rationale: Modo fragmentado para org — Divide buffer em janelas focadas por headline. Alternativa a org-narrow-to-subtree.

Filosofema: *“O foco que se segmenta é o foco que se mantém.”*

```
(use-package org-frakti
  :ensure t)
```

📦 7. Git e Versionamento

📦 magit

Emoji: 📦

Rationale: Interface Git definitiva — staging interativo, rebasing visual, submodule management. 10x mais produtivo que CLI para operações complexas.

Filosofema: *“O git que se vê é o git que se controla.”*

```
(use-package magit
  :ensure t
  :bind
  (("C-x g" . magit-status)
   ("C-x M-g" . magit-dispatch)))
```

📦 magit-delta

Emoji: 📦

Rationale: Deltas syntax-highlighted com cores e diffs side-by-side. Melhora legibilidade de diffs grandes em 10x.

Filosofema: *“O diff que se diferencia é o diff que se lê.”*

```
(use-package magit-delta
  :ensure t
  :config
  (magit-delta-mode +1)
  (setq magit-delta-default-style "two-plus-two"))
```

❑ forge

Emoji: 🛠️

Rationale: Integração GitHub/GitLab em magit — issues, PRs, reviews. Opera com GitHub CLI (gh) como backend.

Filosofema: *“O remote que se gerencia é o remote que se orquestra.”*

```
(use-package forge
  :ensure t
  :after magit)
```

❑ git-gutter

Emoji: 📄

Rationale: Indicadores inline de diff — adições, remoções, modificações. Feedback visual imediato de alterações sem sair do buffer.

Filosofema: *“O risco que se indica é o risco que se mitiga.”*

```
(use-package git-gutter
  :ensure t
  :config
  (global-git-gutter-mode +1)
  (setq git-gutter:update-interval 1
        git-gutter:skip-legend t))
```

❑ 8. AI e Integração

❑ gptel

Emoji: 🗨️

Rationale: Chat com LLMs (OpenAI, Anthropic, Ollama) via API. Buffer conversacional com contexto persistente, streaming, history.

Filosofema: *“A IA que conversa é a IA que se conhece.”*

```
(use-package gptel
  :ensure t
  :config
  (setq gptel-model 'claude-sonnet-4-6
        gptel-backend (gptel-make-anthropic "Claude"
          :stream t
          :key (getenv "ANTHROPIC_API_KEY")))
  :bind
  (("C-c g" . gptel)
   ("C-c G" . gptel-send)))
```

□ simple-httpd

Emoji: 🍷

Rationale: Servidor HTTP em Lisp puro — serve PWA, APIs locais. Base para comunicação Emacs↔browser.

Filosofema: *“O Emacs que serve é o Emacs que expande.”*

```
(use-package simple-httpd
  :ensure t
  :config
  (setq httpd-root "~/ .emacs.d/pwa/"
        httpd-port 7070))
```

□ websocket

Emoji: 🍷

Rationale: WebSocket client/server em Lisp — comunicação bidirecional com PWA/browser. Base para edição interativa remote.

Filosofema: *“O canal que se abre é o canal que se conecta.”*

```
(use-package websocket
  :ensure t)
```

□ epc

Emoji: 🍷

Rationale: RPC server para Python — chamadas assíncronas Lisp↔Python. Bridge para Crawl4AI, parsers complexos, LLMs locais.

Filosofema: *“A ponte que se constrói é a ponte que se atravessa.”*

```
(use-package epc
  :ensure t
  :config
  (setq epc:node-name "emacs-epc"))
```

□ llm

Emoji: 🍷

Rationale: Abstração unificada para LLMs — Ollama, OpenAI, Anthropic. Backends plugáveis com API consistente.

Filosofema: *“A abstração que se uniformiza é a abstração que se simplifica.”*

```
(use-package llm
  :ensure t
  :config
  (setq llm Ollama (make-llm-ollama :chat-model "llama3")))
```

□ llm-ollama

Emoji: 🦙

Rationale: Backend Ollama para llm — LLMs locais (Llama, Mistral, Phi). Privacidade total, custo zero em inferência.

Filosofema: *“O modelo que se hospeda é o modelo que se controla.”*

```
(use-package llm-ollama
  :ensure t
  :after llm)
```

□ 9. Terminal e Shell

□ vterm

Emoji: 📄

Rationale: Emulador de terminal via libvterm — suporte completo a cores, prompts, Completar. 10x mais rápido que term-mode.

Filosofema: *“O terminal que se emula é o terminal que se virtualiza.”*

```
(use-package vterm
  :ensure t
  :config
  (setq vterm-max-scrollback 10000)
  :bind
  (("C-c t" . vterm)
   ("C-c T" . vterm-toggle)))
```

□ eshell

Emoji: 🐚

Rationale: Shell em Elisp — comandos como funções, pipes como listas. Integração nativa com Emacs (buffers, completing-read).

Filosofema: *“O shell que se programa é o shell que se estende.”*

```
(use-package eshell
  :config
  (setq eshell-history-size 10000
        eshell-glob-case-insensitive t)
  (add-hook 'eshell-mode-hook #'eshell/setup-auto-complete))
```

□ direnv

Emoji: 📁

Rationale: Suporte direnv — variáveis de ambiente por diretório. Carrega .envrc automaticamente ao entrar no projeto.

Filosofema: *“O ambiente que se adapta é o ambiente que se respeita.”*

```
(use-package envrc
  :ensure t
  :config
  (envrc-global-mode +1))
```

□ 10. Formatação e Estilo

□ apheleia

Emoji: 📁

Rationale: Formatação automática que não polui undo — formatters externos (black, prettier) com buffer-local preservation. Formata antes de save.

Filosofema: *“O código que se formata é o código que se respeita.”*

```
(use-package apheleia
  :ensure t
  :config
  (apheleia-global-mode +1)
  (setq apheleia-mode-line-lighter " APH"))
```

□ format-all

Emoji: 📁

Rationale: Formatters configuráveis por modo — suporta 100+ linguagens. Integração magit para formatar diffs.

Filosofema: *“O formatador que se configura é o formatador que se adapta.”*

```
(use-package format-all
  :ensure t
  :config
  (format-all-mode +1))
```

□ wgrep

Emoji: 📁

Rationale: Editable grep output — modifications propagam para arquivos.

Batch replace com preview.

Filosofema: *“O grep que se edita é o grep que se transforma.”*

```
(use-package wgrep
  :ensure t
  :config
  (setq wgrep-auto-save-buffer t))
```

✂ crux

Emoji: ✂

Rationale: Coleção de comandos úteis — killing buffer, renaming, swapping windows. Idiomático e consistente.

Filosofema: *“A função que se reutiliza é a função que se otimiza.”*

```
(use-package crux
  :ensure t
  :config
  (crux-mode +1))
```

📄 11. Web e Crawling

📄 eww

Emoji: 📄

Rationale: Navegador web Emacs — páginas estáticas, ebooks, PDFs. Sem dependência de browser externo.

Filosofema: *“O web que se navega é o web que se integra.”*

```
;; Nativo Emacs - configuração
(setq eww-search-prefix "https://duckduckgo.com/?q=")
```

📄 WebSearch

Emoji: 📄

Rationale: Busca web integrada — Google/DuckDuckGo no Emacs. Resultados em buffer navegável.

Filosofema: *“O busca que se pesquisa é o busca que se encontra.”*

```
(use-package websearch
  :ensure t)
```

□ 12. Segurança

□ secrets

Emoji: 🗝️

Rationale: Interface para keyring do OS — GNOME Keyring, KWallet. Armazena API keys de forma segura.

Filosofema: *“O segredo que se guarda é o segredo que se protege.”*

```
(use-package secrets
  :config
  (setq secrets-token-column 5)
  :bind ("C-c k s" . secrets-show-passwords))
```

□ auth-source

Emoji: 🗝️

Rationale: Multi-backend para credentials — ~/.authinfo, pass, secret service. Unifica acesso a tokens.

Filosofema: *“A credencial que se abstrai é a credencial que se simplifica.”*

```
(use-package auth-source
  :config
  (setq auth-sources '("secret-service:login" "~/.authinfo.gpg")))
```

□ 13. Modos Maiores Especiais

□ python

Emoji: 🐍

Rationale: Major mode para Python com tree-sitter. Integração venv, pytest, rope.

Filosofema: *“A cobra que se conhece é a cobra que se doma.”*

```
(use-package python
  :config
  (setq python-indent-offset 4
        python-shell-interpreter "python"))
```

□ typescript

Emoji: 🐍

Rationale: Mode para TypeScript/TSX com tree-sitter. LSP para type-checking, refactoring.

Filosofema: *“O tipo que se anota é o tipo que se garante.”*

```
(use-package typescript-mode
  :ensure t
  :config
  (add-to-list 'auto-mode-alist '("\\.tsx\\'" . typescript-tsx-mode))
  (add-to-list 'eglot-server-programs
    '(typescript-ts-mode . ("typescript-language-server" "--stdio"))))
```

📄 markdown-mode

Emoji: 📄

Rationale: Editing Markdown com preview live. Suporta GFM, table editing, TOC generation.

Filosofema: *“O markdown que se visualiza é o markdown que se publica.”*

```
(use-package markdown-mode
  :ensure t
  :config
  (setq markdown-live-preview-window 'other)
  :bind
  (("C-c C-c p" . markdown-preview-other-window)))
```

📄 gherkin-mode

Emoji: 📄

Rationale: Major mode para BDD Gherkin — syntax highlighting, step completion. Integração com pytest-bdd.

Filosofema: *“O teste que se especifica é o teste que se verifica.”*

```
(use-package gherkin-mode
  :ensure t
  :config
  (add-to-list 'auto-mode-alist '("\\.feature\\'" . gherkin-mode)))
```

📄 solidity-mode

Emoji: 📄

Rationale: Major mode para Solidity — syntax, indentation, flycheck para natspec.

Filosofema: *“O contrato que se verifica é o contrato que se executa.”*

```
(use-package solidity-mode
  :ensure t
  :config
```

```
(add-to-list 'eglot-server-programs
  '(solidity-mode . ("nomicfoundation-solidity-language-server" "--stdio"))))
```

📄 nov

Emoji: 📖

Rationale: Visualizador de ebooks EPUB — rendering interno, navegação por TOC.

Filosofema: *“O livro que se lê é o livro que se compreende.”*

```
(use-package nov
  :ensure t
  :config
  (add-to-list 'auto-mode-alist '(".epub\\'" . nov-mode)))
```

📄 pdf-tools

Emoji: 📖

Emoji: Manipulação de PDFs — search, annotations, outline. Sync de scroll entre PDF e código.

Filosofema: *“O PDF que se anota é o PDF que se estuda.”*

```
(use-package pdf-tools
  :ensure t
  :config
  (pdf-tools-install)
  (setq pdf-view-midnight-colors '("#2e3440" . "#d8dee9")))
```

📄 14. Visualização e Dados

📄 tabular

Emoji: 📊

Rationale: Tabelas em texto — criação, alinhamento, export. Suporta múltiplos delimitadores.

Filosofema: *“O dado que se tabela é o dado que se compara.”*

```
(use-package tabular
  :config
  (setq tab-always-indent 'complete))
```

□ org-table

Emoji: 📊

Rationale: Tabelas avançadas em org — formulas, references, spreadsheets. Calc mode para cálculos.

Filosofema: *“A célula que se referencia é a célula que se atualiza.”*

```
;; Nativo org-mode - configuração
(setq org-table-convert-region-max-lines 1000)
```

□ calc

Emoji: 🧮

Rationale: Calculator científico — algebra, calculus, units. Integração org-babel para computação literate.

Filosofema: *“A matemática que se calcula é a matemática que se prova.”*

```
(use-package calc
  :config
  (setq calc-display-trail nil))
```

□ gnuplot

Emoji: 📈

Rationale: Plotting via Gnuplot — integração org-babel, buffers dedicados. Suporta 2D/3D plots.

Filosofema: *“O gráfico que se plot é o gráfico que se interpreta.”*

```
(use-package gnuplot
  :ensure t)
```

□ graphviz-dot-mode

Emoji: 📄

Rationale: Modo para Graphviz DOT — syntax highlighting, preview inline. Export para SVG/PNG/PDF.

Filosofema: *“O grafo que se desenha é o grafo que se visualiza.”*

```
(use-package graphviz-dot-mode
  :ensure t
  :config
  (setq graphviz-dot-indent-width 2))
```

□ 15. UX e Produtividade

□ org-pomodoro

Emoji: 🍅

Rationale: Timer Pomodoro integrado — notifications, sound, breaks. tracking de tempo por projeto.

Filosofema: *“O tempo que se mede é o tempo que se respeita.”*

```
(use-package org-pomodoro
  :ensure t
  :config
  (setq org-pomodoro-length 25
        org-pomodoro-short-break 5
        org-pomodoro-long-break 15))
```

□ org-cliplink

Emoji: 🔗

Rationale: Insere URLs como títulos — fetch title🔗. Integração org-capture.

Filosofema: *“O link que se titula é o link que se arquiva.”*

```
(use-package org-cliplink
  :ensure t
  :bind
  (("C-c M-l" . org-cliplink-capture)))
```

□ grab-mac-link

Emoji: 🍏

Rationale: Captura links do browser macOS — Chrome, Safari. Insere como org link formatado.

Filosofema: *“O link que se captura é o link que se guarda.”*

```
(use-package grab-mac-link
  :ensure t)
```

□ target

Emoji: 🎯

Rationale: Multiple cursors por regex — targets para operações em batch. Complementa multiple-cursors.

Filosofema: *“O alvo que se define é o alvo que se acerta.”*

```
(use-package target
  :ensure t)
```

□ smart-yank

Emoji: □

Rationale: Kill ring inteligente — de-duplicação, preview antes de yank. Popup de seleção.

Filosofema: *“O que se copia é o que se preserva.”*

```
(use-package smart-yank
  :ensure t
  :config
  (smart-yank-mode +1))
```

□ move-text

Emoji: □

Rationale: Move lines/regions up/down — sem cut/paste. Persiste posição relativa.

Filosofema: *“O texto que se move é o texto que se reorganiza.”*

```
(use-package move-text
  :ensure t
  :config
  (move-text-default-bindings))
```

□ 16. Buffers Especiais e IRs

□ bufler

Emoji: □

Rationale: Gerenciamento de workspaces — agrupa buffers por projeto/modo. Frame layouts persistidos.

Filosofema: *“O workspace que se organiza é o workspace que se navega.”*

```
(use-package bufler
  :ensure t
  :config
  (bufler-mode +1)
  :bind
  (("C-c b" . bufler))
  :bind
  (("C-x C-b" . bufler)))
```

□ sideframe

Emoji: 📄

Rationale: Frames laterais para buffers utility — sidebar de arquivos, IM buffers.

Filosofema: *“O frame que se sidebar é o frame que se economiza.”*

```
(use-package sideframe
  :ensure t)
```

□ popper

Emoji: 📄

Rationale: Popup windows como list — toggle visibility com atalho. Configuração por modo.

Filosofema: *“O popup que se lista é o popup que se controla.”*

```
(use-package popper
  :ensure t
  :config
  (popper-mode +1)
  (setq popper-reference-buffers
    '("\\*Messages\\*" "\\*Compile-Log\\*" "\\*Warnings\\*"))
  :bind
  (("C-c p" . popper-toggle-latest)))
```

□ 17. Integração Python (EPC)

□ pyimport

Emoji: 📄

Rationale: Inserir imports Python automaticamente — resolve ambiguidades, adiciona ao import existente.

Filosofema: *“O import que se completa é o import que se organiza.”*

```
(use-package pyimport
  :ensure t)
```

□ pytest

Emoji: 📄

Rationale: Integração pytest — run tests, navigate errors, coverage. Feedback rápido durante TDD.

Filosofema: *“O teste que se automatiza é o teste que se confia.”*

```
(use-package pytest
  :ensure t
  :config
  (setq pytest-project-root nil))
```

□ elpy

Emoji: 🐍

Rationale: Python IDE completo — rope, flymake, shell, virtualenv. Alternativa a eglot para setups complexos.

Filosofema: *“O IDE que se integra é o IDE que se completa.”*

```
(use-package elpy
  :ensure t
  :config
  (elpy-enable)
  (when (executable-find "ruff")
    (setq elpy-formatter 'elpy-formatter-ruff)))
```

□ 18. Testing e Debug

□ dape

Emoji: 🐛

Rationale: Debug Adapter Protocol implementation — VS Code debuggers em Emacs. Suporta Go, Python, Node, Rust.

Filosofema: *“O bug que se depura é o bug que se compreende.”*

```
(use-package dape
  :ensure t
  :config
  (dape-mode +1)
  :bind
  (("C-c d d" . dape)
   ("C-c d b" . dape-breakpoint-toggle)))
```

□ EBT

Emoji: 🧪

Rationale: Emacs Batch Testing — test runner para qualquer framework. Results em buffer dedicated.

Filosofema: *“O teste que se batch é o teste que se escala.”*

```
(use-package ebt
  :ensure t)
```

□ 19. Funcionalidades Avançadas (Futuro)

□ ekg

Emoji: □

Rationale: Extended Knowledge Graph — SQLite nativo, multi-□ nodes, queries estruturadas. Sucessor programável do org-brain.

Filosofema: *“O knowledge que se consulta é o knowledge que se descobre.”*

```
;; Configuração futura
(use-package ekg
  :ensure t
  :config
  (setq ekg-database-directory "~/emacs.d/ekg/"))
```

□ memory-graph

Emoji: □

Rationale: Visualização de estrutura de memória do projeto — dependências, complexidade, code coverage.

Filosofema: *“O grafo que se memoriza é o grafo que se compreende.”*

□ temporal-nav

Emoji: □

Rationale: Navegação temporal no git — timeline de commits, blame contextual, time-lapse.

Filosofema: *“O tempo que se navega é o tempo que se□□.”*

□ corporal-mode

Emoji: □

Rationale: Feedback háptico/visual para estados do editor — vibração, sons contextuais, RGB.

Filosofema: *“O corpo que se integra é o corpo que se comunica.”*

□ nnc

Emoji: 🧠

Rationale: Neural Network Completion — LLM local para sugestões contextuais avançadas.

Filosofema: “A rede que se completa é a rede que se antecipa.”

□ 20. Dependências Python (Instalar Separadamente)

□ Stack de Data Science

```
pip install --break-system-packages \
    numpy \
    pandas \
    sentence-transformers \
    pyarrow \
    networkx \
    rank-bm25
```

□ Web Scraping

```
pip install --break-system-packages \
    crawl4ai \
    playwright \
    beautifulsoup4 \
    lxml
```

□ AI/LLM

```
pip install --break-system-packages \
    anthropic \
    openai \
    ollama
```

□ PDF Processing

```
pip install --break-system-packages \
    pymupdf \
    pypdf2 \
    magic-pdf # RAG-Anything/MinerU
```

Outline II: Relacionamentos entre Bibliotecas

1. Cadeia de Dependência Funcional

org-roam → org-roam-ui

Emoji: →

Rationale: org-roam gerencia nodes; org-roam-ui visualiza o grafo resultante. Dados fluem de node para visualização.

Filosofema: *“O grafo que se guarda é o grafo que se mostra.”*

org-roam (dados) → org-roam.db → org-roam-ui (visualização)

eglot → treesit-auto → flymake

Emoji: → →

Rationale: treesit-auto fornece parsers; eglot usa parsers para LSP; flymake exibe diagnostics do LSP.

Filosofema: *“O parser que se adapta é o parser que se integra.”*

treesit-auto (grammar) → eglot (LSP) → flymake (UI diagnostics)

projectile → treemacs → imenu-list

Emoji: → →

Rationale: projectile detecta projeto; treemacs mostra file tree do projeto; imenu-list mostra symbols do arquivo atual.

Filosofema: *“A navegação que se hierarquiza é a navegação que se domina.”*

projectile (projeto) → treemacs (files) + imenu-list (symbols)

consult → embark → wgrep

Emoji: → →

Rationale: consult busca e mostra resultados; embark oferece ações sobre resultados; wgrep permite editing in-place.

Filosofema: *“O busca que se actua é o busca que se transforma.”*

consult (busca) → embark (ações) → wgrep (edição)

📄 company → 📄 yasnippet → 📄 copilot

Emoji: 📄 → 📄 → 📄

Rationale: company é o framework de completion; yasnippet e copilot são backends que se complementam — templates + AI suggestions.

Filosofema: *“A completion que se complementa é a completion que se completa.”*

company (framework) ← [yasnipet, copilot] (backends)

📄 magit → 📄 magit-delta → 📄 forge

Emoji: 📄 → 📄 → 📄

Rationale: magit opera git; magit-delta embeleza diffs; forge adiciona GitHub/GitLab integration.

Filosofema: *“O git que se eleva é o git que se orchestra.”*

magit (core) → magit-delta (display) + forge (remote)

📄 gptel → 📄 epc → 📄 simple-httpd

Emoji: 📄 → 📄 → 📄

Rationale: gptel usa LLMs via API; epc conecta com Python para tools externas (crawl4ai); simple-httpd serve PWA que interage com Emacs.

Filosofema: *“A IA que se estende é a IA que se amplifica.”*

gptel (LLM) ← epc (Python tools) ← simple-httpd (PWA server)

📄 org-roam → 📄 org-super-agenda → 📄 org-transclusion

Emoji: 📄 → 📄 → 📄

Rationale: org-roam fornece estrutura de notas; org-super-agenda mostra views customizadas; org-transclusion inclui conteúdo entre notas.

Filosofema: *“O org que se vista é o org que se reutiliza.”*

org-roam (notas) → org-super-agenda (views) + org-transclusion (includes)

📄 eww → 📄 epc → 📄 crawl4ai

Emoji: 📄 → 📄 → 📄

Rationale: eww navega páginas simples; crawl4ai (via EPC) processa páginas JS-heavy; resultados vão para org-roam.

Filosofema: *“O crawl que se integra é o crawl que se knowledge.”*

eww (pages) + crawl4ai via epc → org-roam (captura)

☐ gptel → ☐ llm → ☐ llm-ollama

Emoji: ☐ → ☐ → ☐

Rationale: gptel é cliente conversacional; llm é abstração de backends; llm-ollama adiciona suporte a Ollama local.

Filosofema: *“A abstração que se instancia é a abstração que se realiza.”*

gptel (UI) → llm (abstraction) → llm-ollama (backend)

☐ doom-modeline → ☐ which-key → ☐ posframe

Emoji: ☐ → ☐ → ☐

Rationale: doom-modeline exibe status; which-key mostra bindings; posframe renderiza popups всплывающие.

Filosofema: *“A interface que se notifica é a interface que se comunica.”*

doom-modeline (status) + which-key (help) + posframe (popups)

☐ flymake → ☐ flymake-vale → ☐ vale

Emoji: ☐ → ☐ → ☐

Rationale: flymake é framework de linting; flymake-vale é backend para prosa; vale é o linter externo.

Filosofema: *“O lint que se estende é o lint que se verifica.”*

flymake (framework) → flymake-vale (prose) + [language linters]

☐ vterm → ☐ eshell → ☐ envrc

Emoji: ☐ → ☐ → ☐

Rationale: vterm para terminal real; eshell para shell scriptável; envrc para environment por diretório.

Filosofema: *“O shell que se adapta é o shell que se respeita.”*

envrc (environment) → vterm/eshell (shells)

☐ apheleia → ☐ format-all → ☐ graphviz-dot-mode

Emoji: ☐ → ☐ → ☐

Rationale: apheleia formata no save; format-all suporta mais linguagens;

graphviz-dot-mode formata DOT graphs.

Filosofema: “O formato que se preserva é o formato que se mantém.”

format-all (formatters) ← apheleia (trigger) + graphviz (DOT)

📄 org-table → 📄 calc → 📄 gnuplot

Emoji: 📄 → 📄 → 📄

Rationale: org-table para dados tabulares; calc para cálculos; gnuplot para visualização. Pipeline de dados completo.

Filosofema: “O dado que se processa é o dado que se revela.”

org-table (data) → calc (compute) → gnuplot (visualize)

📄 dape → 📄 eglot

Emoji: 📄 → 📄

Rationale: dape usa DAP (Debug Adapter Protocol); eglot usa LSP; ambos derivam de project analysis.

Filosofema: “O debug que se adapta é o debug que se integra.”

dape (debugging) ↔ eglot (analysis) – ambos project-aware

📄 ekg ↔ 📄 org-roam

Emoji: 📄 ↔ 📄

Rationale: ekg é sucessor programável de org-brain; pode coexistir com org-roam para diferentes use-cases.

Filosofema: “O knowledge que se evolui é o knowledge que se adapta.”

ekg (futuro) + org-roam (presente) – conhecimento dual-track

📄 bufler → 📄 sideframe → 📄 popper

Emoji: 📄 → 📄 → 📄

Rationale: bufler gerencia workspaces completos; sideframe cria frames laterais; popper toggle popups. Hierarquia de window management.

Filosofema: “A janela que se gerencia é a janela que se organiza.”

bufler (workspaces) → sideframe (sidebar) + popper (popup)

□ 2. Fluxos de Dados Entre Bibliotecas

Fluxo: Captura de Web → Conhecimento

Browser (captura URL)
↓ org-protocol / grab-mac-link
org-roam (node criado com metadados)
↓
crawl4ai via epc (parsing + embeddings)
↓
Knowledge Base SQLite (kg.db)
↓
RAG Pipeline (steps 1-10)
↓
gptel (contexto disponível para AI)

Fluxo: Editing → LSP → Diagnostics → UI

Arquivo editado
↓ treesit-auto (parse)
eglot (análise semântica)
↓
flymake / lsp-ui (diagnostics)
↓
doom-modeline (indicadores)
↓
which-key / embark (ações de correção)

Fluxo: Prompt → RAG → Code Generation

Prompt do usuário (via gptel)
↓
Step 4: Expansão (CoT via llm)
↓
Step 6: Recuperação híbrida (consult-ripgrep + semantic search)
↓
Knowledge Base (SQLite + vetores)
↓
Step 7-9: Purificação + Geração + Verificação
↓
Output para buffer
↓
magit (commit se aprovado)

Fluxo: TDD Cycle

```
pytest (teste falha)
  ↓
projectile / consult (navega para código)
  ↓
eglot + company (implementa)
  ↓
apheleia (formata)
  ↓
magit (stage + commit)
  ↓
pytest (teste passa)
  ↓
doom-modeline (indicador verde)
```

Fluxo: Knowledge Management

```
Chatlog (capturado via gptel)
  ↓
MAGMA destilation (via epc + Python)
  ↓
org-roam (node com TIP0=chatlog)
  ↓
org-brain (conceptualização)
  ↓
Knowledge Base (entidades + relações)
  ↓
org-super-agenda (visão integrada)
```

□ 3. Map de Ativações e Mutual Indexing

Ativação por Modo Maior

Modo	Bibliotecas Ativas
python-ts-mode	eglot, treesit-auto, flymake, company, pytest, elpy
typescript-ts-mode	eglot, treesit-auto, flymake, company, apheleia
org-mode	org-roam, org-brain, org-super-agenda, org-transclusion, gptel
emacs-lisp-mode	elisp-indent, checkdoc, helpful, rainbow-delimiters
magit-status-mode	magit, magit-delta, forge, git-gutter
eshell-mode	eshell, vterm, envrc
eww-mode	eww, org-protocol

Mutual Indexing: Bidirectional Links

Código fonte (file:line)
↓ [Âncora]
Knowledge Base (node_id)
↓ [Embedding]
Vector Store (semantic search)
↓ [FTS]
Full-text Index (keyword search)
↓ [Links]
org-roam / org-brain (conceitos)

□ 4. Arquitetura de Comunicação

Elisp ↔ Python (EPC)

Elisp (Emacs)
↓ epc:call-deferred
EPC Server (Python)
↓
├─ crawl4ai (web scraping)
├─ sentence-transformers (embeddings)
├─ anthropic (LLM calls)
└─ RAG tools (indexing, retrieval)

Emacs ↔ Browser (WebSocket + HTTP)

Emacs (simple-httpd)
↓ HTTP :7070
PWA (browser)
↓ WebSocket :7071
Emacs (websocket.el)
↓
├─ IR Graph Viewer (Cytoscape.js)
├─ Org-Roam UI (alternativa)
└─ Custom dashboards

□ 5. Grupos de Co-instalação (Pacotes Recomendados)

Grupo: Core IDE

- use-package, which-key, posframe

- avy, windmove, smartparens, rainbow-delimiters
 - projectile, treemacs, imenu-list, dumb-jump
 - eglot, treesit-auto, flymake
 - company, yasnippet
 - magit, magit-delta, git-gutter
 - doom-modeline, modus-themes
-

Grupo: Org e Conhecimento

- org-roam, org-roam-ui
 - org-brain
 - org-super-agenda
 - org-transclusion
 - org-download
-

Grupo: AI e RAG

- gptel, llm, llm-ollama
 - simple-httpd, websocket, epc
 - company (para completion em buffers AI)
-

Grupo: Web e Crawling

- eww (nativo)
 - epc + Crawl4AI (via Python EPC)
 - org-protocol (captura links)
-

Grupo: Produtividade

- consult, embark
 - apheleia, format-all
 - vterm, eshell, envrc
 - org-pomodoro
 - bufler, popper
 - crux, move-text
-

Grupo: Linguagens Específicas

- **Python:** elpy, pytest, pyimport
- **TypeScript:** typescript-mode, tsx-mode

- **Markdown:** markdown-mode
 - **BDD:** gherkin-mode
 - **Smart Contracts:** solidity-mode
 - **PDFs:** nov, pdf-tools
-

Plano de Implementação Pragmático

Fase 1: Fundação (Dia 1-2)

1. Instalar `straight.el` e `use-package`
2. Configurar `which-key`, `posframe`
3. Setup `modus-themes`, `doom-modeline`
4. Instalar `all-the-icons`

Resultado: Emacs com UI funcional e discoverável

Fase 2: Edição e Navegação (Dia 3-4)

1. `avy`, `windmove` para navegação
2. `smartparens`, `rainbow-delimiters` para edição
3. `projectile`, `treemacs` para projeto
4. `consult`, `embark` para busca
5. `imenu-list` para symbols

Resultado: Navegação 10x mais rápida

Fase 3: Code Intelligence (Dia 5-6)

1. `eglot` + `treesit-auto`
2. `flymake` + LSP diagnostics
3. `company` + `yasnipet`
4. `xref` configurado

Resultado: Autocomplete e jump-to-definition

Fase 4: Git e Versionamento (Dia 7)

1. `magit` + `magit-delta`
2. `git-gutter`
3. `forge` (se usar GitHub/GitLab)

Resultado: Git workflow completo

Fase 5: Org e Conhecimento (Dia 8-10)

1. org-roam + org-roam-ui
2. org-brain
3. org-super-agenda
4. Templates de capture

Resultado: Second brain funcional

Fase 6: AI Integration (Dia 11-14)

1. gptel com Anthropic/OpenAI
2. llm + llm-ollama
3. simple-httpd + websocket para PWA
4. epc para Python bridge

Resultado: AI companion integrado

Fase 7: Polish e Avançado (Dia 15+)

1. bufler para workspaces
 2. apheleia para formatting
 3. org-pomodoro para time tracking
 4. Web scraping setup (crawl4ai via EPC)
 5. Customizações avançadas
-

Priorização por Impacto

Biblioteca	Impacto	Esforço	Prioridade
use-package	☐ Alto	☐ Baixo	P0
consult + embark	☐ Alto	☐ Baixo	P0
projectile + treemacs	☐ Alto	☐ Baixo	P0
eglot + treesit-auto	☐ Alto	☐ Médio	P0
magit	☐ Alto	☐ Baixo	P0
org-roam	☐ Alto	☐ Médio	P1
gptel	☐ Alto	☐ Médio	P1
company + yasnippet	☐ Médio	☐ Baixo	P1
doom-modeline	☐ Médio	☐ Baixo	P2
org-brain	☐ Médio	☐ Médio	P2
vterm	☐ Médio	☐ Baixo	P2

Biblioteca	Impacto	Esforço	Prioridade
apheleia	☐ Médio	☐ Baixo	P2
epc + Crawl4AI	☐ Médio	☐ Médio	P3
bufler	☐ Médio	☐ Médio	P3

Documento gerado em: 2026-04-16

Versão: 1.0.0

*Fontes: ~/emacs/***